



**Département d'informatique
et de mathématique**
Université du Québec à Chicoutimi

**Département d'Informatique et de Mathématique (DIM)
Architecture des applications d'entreprise (8INF853)
TP**

Professeur : Hamid Mcheick	Trois étudiants pour ce TP2
Semestre : Été 2026	Date de distribution : 07 mai 2026
Pondération : 20 points	Date de remise : 02 juin 2025

Le but du TP est de familiariser l'étudiant avec les concepts suivants:

- La modularité, les couches architecturaux, les vues d'une architecture
- L'interaction entre les composantes logicielles
- La réduction de couplage des composants pour faciliter l'adaptation de l'architecture
- Les styles/patrons architecturaux, protocoles d'interactions entre composantes
- La conception architecturale
- Les méthodes d'analyse architecturales
- La méthode de conception architecturale ADD
- La dépendance des composants logiciels

Vous devez faire **deux** questions seulement parmi les questions suivantes.

Question 1 (50 points)

a) Contexte

Le but de ce travail est de faire la conception architecturale d'un système informatique.

L'élicitation des exigences et l'analyse ont déjà été complétées, et sont présentées dans les documents suivants :

- Les cas d'utilisation, disponibles dans le document suivant [cas d'utilisation](#)
- Le [Modèle objet, format Visio](#).
- Les diagrammes de séquence systèmes. Vous trouverez le document ici: [diagrammes de séquence de EURent](#).
- Les opérations système. Vous trouverez le document ici: [Opérations système EURent](#)
- Un document complémentaire d'exigences architecturales, [Exigences Architecturales EURent](#).

b) Objectifs

Le but de ce travail est de faire la conception architecturale du système selon la méthode *Attribute-Driven Design* présentée en classe. Pour ce faire, vous devriez :

1. faire l'*analyse architecturale du système*, qui consiste, en quelque sorte, à concevoir l'*architecture fonctionnelle* du système. Ceci consiste à identifier les paquetages (package) fonctionnels du niveau analyse. Ces paquetages nous serviront de guide/point de départ pour la décomposition sous-jacente à la méthode ADD.
2. faire la conception architecturale, proprement dite, grâce à la méthode ADD (*attribute-driven design*), en veillant à documenter les composantes retenues.

c) Livrables

Un document Word ou pdf (copie électronique) qui contient,

- 1) la description des paquetages d'analyse, 35%
- 2) une trace de l'application de la méthode ADD, qui justifie les choix architecturaux, et 35%
- 3) la documentation des composantes retenues. 30%

Pour ce faire, vous vous servirez du [gabarit](#) suivant.

d) Exemple de solution

Vous pouvez consulter [l'énoncé du TP1 d'une session passée](#). Bien que le document d'analyse fasse une centaine de pages, le plus important à ce stade-ci est le modèle de classes et la liste des cas d'utilisation.

Question 2 (50 points)

Le but de cette question de la description et la comparaison de deux méthodes d'analyses d'architectures logicielles : *Architecture Trade-off Analysis Methode (ATAM)*, et *architecture-Level Modifiucability analysis (ALMA)*.

Vous reprendrez chacun de principes de ces deux méthodes puis vous y apporterez votre propre analyse en les comparants selon les critères suivants :

- La qualité évaluée
- La description des processus
- Leurs forces
- Leur application pour différents types de systèmes

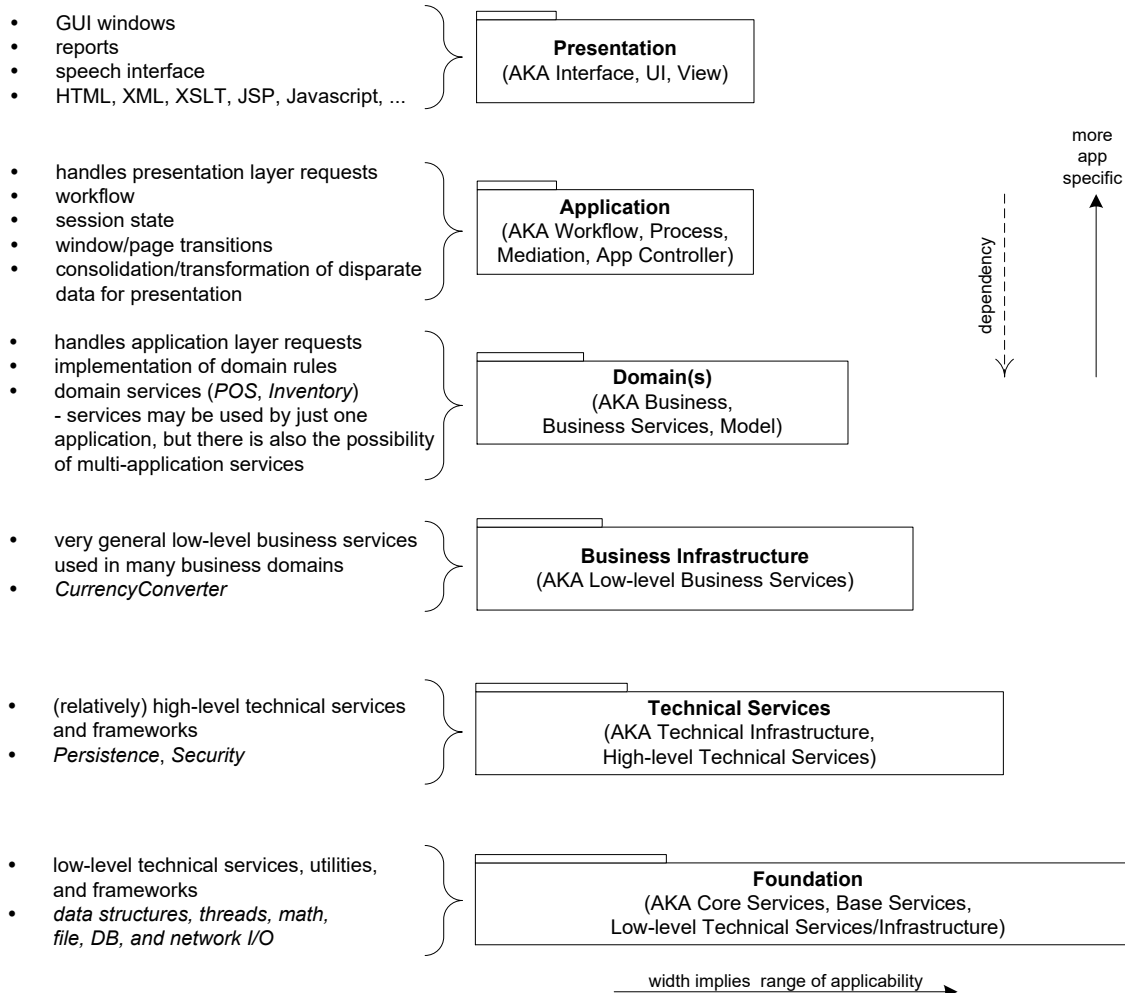
Livrables

- Copie électronique (word ou pdf).
- Description de ATAM : 30 %
- Description de ALMA : 30%
- Comparaison : 40%

Question 3 (50 points)

L'architecture de logicielle permet d'identifier des exigences fonctionnelles et non-fonctionnelles qui influencent la conception d'un système. Cependant, l'architecture a plusieurs problèmes : couplage et non modularité du système rendent la mise à niveau, la mise à jour et l'adaptation d'un logiciel à des nouvelles technologies difficiles, instabilité du code fortement couplé, etc. Une solution est d'organiser la structure logique à grande-échelle d'un logiciel en couches distinctes, cohésives et modulaires. Une façon d'organiser l'architecture en couches est donnée dans la figure suivante.

- a) Expliquez brièvement le patron architectural en couche : problème, solution, discussion
- b) Expliquez le rôle de chaque couche donnée ici-bas dans la figure
- c) Donnez un exemple de package de chaque couche
- d) Expliquez comment ces couches aident à rendre l'architecture modulaire et cohésive



Livrables

- Copie électronique.
- Le barème est le suivant :
- Description des couches de l'architecture : 40 points sur 100
- Exemple de package de chaque couche : 40 points sur 100
- Comment aider à avoir une architecture modulaire : 20 points sur 100

Question 4 (50 points)

Considérons la méthode d'ingénierie informatique DevOps ou l'architecture Micro-services, qui se caractérise par la promotion de l'automatisation et de toutes les étapes de la création d'un logiciel.

- Expliquez le rôle de cette méthode
- Développer une application quelconque (ex. bancaire) en utilisant cette méthode DevOps ou architecture Micro-services : identifiez les micro-services, donnez les diagrammes de micro-services et leurs implémentations.
- Donnez les avantages et les inconvénients de cette méthode ou architecture ? expliquez

Livrables

- Copie électronique.
- Le barème est le suivant :
- Description de rôle et étapes de cette méthode : 30 points sur 100
- Application complète et exécutable : 40 points sur 100
- Avantages et inconvénients : 30 points sur 100

Question 5 (50 points)

Le langage objet Java fournit un mécanisme de packages en modules (*Java platform module : Java module system 9+*).

- Expliquez brièvement le fonctionnement de cette *plateforme avec des figures*
- Appliquez cette plateforme sur un exemple complet du code (application) pour montrer le mécanisme d'adaptation dynamique de comportements. Montrez cet exemple (code complet) utilisant ce mécanisme.
- Quels sont les avantages et les inconvénients de cette *plateforme* ?

Livrables

- Copie électronique.
- Le barème est le suivant :
- Description de *la plateforme* : 40 points sur 100
- Exemple : 40 points sur 100
- Avantages et inconvénients de cette plateforme : 20 points sur 100

Question 6 (50 points)

Implémentation de patron architectural publisher-subscriber (Observer/Observable au niveau conception) avec la programmation par aspect (AspectJ).

Il s'agit de développer un programme d'interaction entre deux ou plusieurs composantes (publishers et subscribers) en se basant sur le patron Observer/Obervable. Ce programme implémente le protocole d'interaction entre ces composantes dans un module (composant). Il permet de simuler leur relation « dépendants » afin de réduire le couplage entre ces composants.

La programmation par aspect, une de méthodes de séparation des préoccupations, permet d'adapter des applications (ou composantes) et de réaliser un protocole d'interaction. La programmation par aspects propose de structurer les programmes en les décomposant en aspects et classes. Un aspect est une unité de décomposition transversale au découpage de l'application en classes. Un aspect peut avoir une ou plusieurs propriétés: points de jonction, introduction des méthodes, des attributs, et des classes, etc.). AspectJ est une extension de Java pour la programmation par aspect. Vous pouvez utiliser AspectJ avec Elcipse, Netbeans, IntelliJ ou autres outils pour développer votre programme.

Livrables

- Copie électronique (word ou pdf).
- Description de l'architecture : 30%
- Étude de cas (implémentation complète des composants et dependants) : 40%
- Forces et faiblesses de ce patron architectural : 30%